

LANGCHAIN

From Basics to Advanced

A Complete, Fresher-Friendly Guide to the LangChain Ecosystem

Covers core concepts, package structure & dependencies, LCEL, RAG pipelines, memory, tools, agents, LangGraph, LangSmith, and deployment — with runnable code examples.

Prepared for: Aspiring LLM / GenAI Application Developers

Level: Beginner → Advanced

Format: Concept Guide + Code Walkthroughs

Table of Contents

1. What Is LangChain?	3
2. The LangChain Ecosystem & Package Structure	4
3. Installation & Environment Setup	6
4. Core Building Blocks (Beginner Level)	7
4.1 Chat Models & LLMs	7
4.2 Prompt Templates	7
4.3 Output Parsers	7
4.4 LCEL — the LangChain Expression Language	8
5. Intermediate Concepts	9
5.1 Memory — giving a chain conversation history	9
5.2 Document Loaders & Text Splitters	9
5.3 Embeddings & Vector Stores	9
5.4 Retrievers & a Full RAG Pipeline	10
6. Advanced Concepts	11
6.1 Tools & Tool Calling	11
6.2 Agents — models that decide their own steps	11
6.3 LangGraph — stateful, multi-step orchestration	11
6.4 Streaming & Async	12
6.5 LangSmith — observability, tracing & evaluation	12
6.6 Deployment	12
7. Putting It Together: A Minimal RAG Chatbot	14
8. Best Practices & Common Pitfalls	16
9. Learning Roadmap	17
10. Glossary	18
11. Further Resources	19

1. What Is LangChain?

LangChain is an open-source **framework for building applications powered by large language models (LLMs)**. On its own, an LLM is just a function that turns text into text. LangChain provides the plumbing around that function: standard interfaces for prompts, models, memory, tools, retrieval, and multi-step orchestration — so you don't have to reinvent it for every provider (OpenAI, Anthropic, Google, local models, etc.) or every use case.

1.1 Why does LangChain exist?

Calling an LLM API directly is easy for a single prompt-in, text-out request. Real applications usually need more:

- Combining several LLM calls into a **pipeline** (e.g. summarize → translate → format).
- Giving the model access to **your own data** (PDFs, databases, websites) — this is *Retrieval-Augmented Generation (RAG)*.
- Letting the model remember earlier turns of a conversation (**memory**).
- Letting the model call external **tools** (search, calculators, APIs, code execution).
- Swapping model providers without rewriting application logic.
- Tracing, debugging, and evaluating what a chain of LLM calls actually did.

LangChain standardizes all of this behind consistent interfaces, so the same code pattern works whether you're calling GPT-4o, Claude, Gemini, or a local Llama model.

1.2 Where LangChain fits in a GenAI application

NOTE

Mental model: your LLM provider (OpenAI, Anthropic, ...) supplies the **model**. LangChain supplies the **scaffolding** around it — prompts, chains, memory, retrieval, tools and agents — so you can compose a real application instead of one raw API call.

1.3 Who is LangChain for?

- Developers building **chatbots**, **Q&A over documents**, **coding assistants**, or **autonomous agents**.
- Teams that need to be **model-agnostic** (easy to switch between OpenAI/Anthropic/etc.).
- Anyone prototyping quickly, then productionizing with observability (LangSmith) and deployment tooling.

2. The LangChain Ecosystem & Package Structure

This is the part that confuses most freshers: **"LangChain" is not one package** — it's a family of packages that depend on each other in a specific layered way. Understanding this layout is the key to understanding LangChain as a dependency in your project.

2.1 The package layers

Package	Role	Typical dependency
langchain-core	The foundation: base abstractions — Runnable, messages, prompt templates, output parsers, base classes for chat models & vector stores. Almost no third-party dependencies.	Installed automatically as a dependency of langchain
langchain	Higher-level orchestration: chains, agents, retrieval strategies, generic building blocks that are provider-agnostic.	pip install langchain
langchain-community	Community-maintained integrations: hundreds of document loaders, vector stores, tools, and third-party model wrappers in one place.	pip install langchain-community
Partner packages e.g. langchain-openai, langchain-anthropic, langchain-google-gemini	Dedicated, maintained-by-the-provider integration packages. Preferred over the generic community versions for production use.	pip install langchain-openai
langgraph	Low-level orchestration engine for building stateful, multi-step, cyclical LLM applications (agents, multi-agent systems). The modern way to build agents.	pip install langgraph
langsmith	Observability & evaluation platform: tracing, debugging, dataset-based testing, prompt versioning for anything built with LangChain (or even without it).	pip install langsmith
langserve	Helper for exposing a LangChain Runnable/chain as a REST API using FastAPI. Useful for quick deployment of a chain as a service.	pip install "langserve[all]"

2.2 How the dependency graph looks

```

your-application
├── langchain                (chains, agents, retrieval helpers)
│   ├── langchain-core      (Runnable, prompts, messages, parsers)
│   ├── langchain-openai    (ChatOpenAI, OpenAIEmbeddings)
│   │   └── langchain-core
│   └── langchain-community (loaders, misc vector stores, tools)
│       └── langchain-core
├── langgraph                (stateful graphs / modern agents)
│   └── langchain-core
└── langsmith                (tracing / eval - optional but recommended)
  
```

TIP

Key takeaway: `langchain-core` is the shared foundation almost everything else builds on. When you see a version-mismatch error, it's usually because `langchain-core` got out of sync with `langchain` or a partner package. Keep them updated together.

2.3 A typical, minimal install

```

# Core framework + OpenAI integration + a vector store
pip install langchain langchain-openai langchain-community

# For RAG you'll usually also want:
pip install chromadb tiktoken pypdf

# For agents / multi-step workflows:
pip install langgraph

# For tracing & debugging (highly recommended once past "hello world"):
pip install langsmith
  
```

2.4 Why the split happened

Early LangChain shipped every integration in one giant package, which made installs heavy and released constantly-breaking changes whenever any single provider updated its SDK. Splitting into `langchain-core` (stable base), `langchain` (orchestration), `langchain-community` (everything else), and dedicated partner packages keeps your dependency tree smaller and versioning saner — you only install what you actually use.

3. Installation & Environment Setup

3.1 Prerequisites

- Python 3.9+ (3.10/3.11 recommended)
- A virtual environment (venv, conda, or poetry) — never install globally
- An API key for at least one model provider (OpenAI, Anthropic, Google, etc.), or a local model via Ollama

3.2 Setting up a project

```
# 1. Create and activate a virtual environment
python -m venv .venv
.venv\Scripts\activate          # Windows
# source .venv/bin/activate     # macOS/Linux

# 2. Install the packages you need
pip install langchain langchain-openai python-dotenv

# 3. Store your API key in a .env file (never hard-code it)
echo OPENAI_API_KEY=sk-... > .env
```

3.3 Loading credentials safely

```
from dotenv import load_dotenv
load_dotenv()  # reads .env and sets environment variables

from langchain_openai import ChatOpenAI
llm = ChatOpenAI(model="gpt-4o-mini")  # picks up OPENAI_API_KEY automatically
```

CAUTION

Never commit .env files or API keys to version control. Add .env to your .gitignore.

4. Core Building Blocks (Beginner Level)

Every LangChain application is built from a small set of composable pieces. Learn these five and you can read almost any LangChain codebase.

4.1 Chat Models & LLMs

A **chat model** wraps a provider's API behind a common interface: you send a list of *messages* (system / human / AI) and get an AI message back.

```
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage, SystemMessage

llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.2)

response = llm.invoke([
    SystemMessage(content="You are a concise coding assistant."),
    HumanMessage(content="What does a Python virtual environment do?"),
])
print(response.content)
```

4.2 Prompt Templates

Instead of hand-building strings, **PromptTemplate** / **ChatPromptTemplate** lets you define reusable prompts with variables — critical once prompts get long or are reused across a codebase.

```
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant that explains topics to {audience}."),
    ("human", "Explain {topic} in 3 sentences."),
])

messages = prompt.invoke({"audience": "beginners", "topic": "vector databases"})
```

4.3 Output Parsers

LLMs return text. Output parsers turn that text into structured data your code can use — plain strings, JSON, or a typed Pydantic model.

```
from langchain_core.output_parsers import StrOutputParser, JsonOutputParser
from pydantic import BaseModel, Field

class MovieReview(BaseModel):
    title: str = Field(description="Movie title")
    sentiment: str = Field(description="positive, negative, or neutral")
    score: int = Field(description="1-10 rating")

parser = JsonOutputParser(pydantic_object=MovieReview)
```

4.4 LCEL — the LangChain Expression Language

Modern LangChain composes components with the pipe operator `|`. Each component (prompt, model, parser, retriever, ...) implements a common **Runnable** interface, so you can chain them like Unix pipes. This replaces most of the older, more verbose Chain subclasses.

```
chain = prompt | llm | StrOutputParser()

result = chain.invoke({"audience": "beginners", "topic": "vector databases"})
print(result)

# Every Runnable automatically supports:
chain.invoke(...)      # single call
chain.batch([...])     # many calls in parallel
chain.stream(...)       # token-by-token streaming
await chain.ainvoke(...) # async
```

TIP

LCEL is the single most important concept to internalize early: prompts, models, retrievers, parsers and even whole sub-chains are all **Runnable** objects that compose the same way with `|`.

5. Intermediate Concepts

5.1 Memory — giving a chain conversation history

By default, each `invoke()` call is stateless. To build a chatbot that remembers earlier turns, you attach message history to the chain.

```
from langchain_core.chat_history import InMemoryChatMessageHistory
from langchain_core.runnables.history import RunnableWithMessageHistory

store = {}
def get_history(session_id: str):
    if session_id not in store:
        store[session_id] = InMemoryChatMessageHistory()
    return store[session_id]

chat_with_memory = RunnableWithMessageHistory(chain, get_history)
chat_with_memory.invoke(
    {"topic": "your question"},
    config={"configurable": {"session_id": "user-123"}},
)
```

For production apps, chat history is usually persisted in Redis, Postgres, or a similar store instead of an in-memory dict.

5.2 Document Loaders & Text Splitters

To let an LLM answer questions about *your* data, you first load it, then split it into chunks small enough to fit a model's context window and to keep retrieval precise.

```
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

loader = PyPDFLoader("handbook.pdf")
docs = loader.load()

splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=150)
chunks = splitter.split_documents(docs)
```

5.3 Embeddings & Vector Stores

An **embedding model** converts text into a numeric vector that captures its meaning. A **vector store** indexes those vectors so you can search by semantic similarity instead of exact keyword match.

```
from langchain_openai import OpenAIEmbeddings
from langchain_chroma import Chroma

embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
vectorstore = Chroma.from_documents(chunks, embedding=embeddings)
```

5.4 Retrievers & a Full RAG Pipeline

A **retriever** wraps a vector store to fetch the most relevant chunks for a query. Combined with a prompt and a model, this is **Retrieval-Augmented Generation (RAG)** — the pattern behind most "chat with your documents" applications.

```
from langchain_core.runnables import RunnablePassthrough
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

retriever = vectorstore.as_retriever(search_kwargs={"k": 4})

rag_prompt = ChatPromptTemplate.from_template(
    "Answer the question using ONLY the context below.\n\n"
    "Context:\n{context}\n\nQuestion: {question}"
)

def format_docs(docs):
    return "\n\n".join(d.page_content for d in docs)

rag_chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | rag_prompt
    | llm
    | StrOutputParser()
)

print(rag_chain.invoke("What is the refund policy?"))
```

NOTE

This five-piece pattern — *load* → *split* → *embed* → *retrieve* → *generate* — is worth memorizing. It's the backbone of the vast majority of real-world LangChain applications.

6. Advanced Concepts

6.1 Tools & Tool Calling

A **tool** is a Python function the model can choose to call — a calculator, a web search, a database query, an internal API. Modern chat models support native **tool calling**: you describe the function, the model decides when to call it and with what arguments.

```
from langchain_core.tools import tool

@tool
def get_weather(city: str) -> str:
    """Get the current weather for a given city."""
    return f"It is sunny in {city}."

llm_with_tools = llm.bind_tools([get_weather])
ai_msg = llm_with_tools.invoke("What's the weather in Hyderabad?")
print(ai_msg.tool_calls)  # [{'name': 'get_weather', 'args': {'city': 'Hyderabad'}, ...}]
```

6.2 Agents — models that decide their own steps

An **agent** repeatedly lets the model choose: answer directly, or call a tool and observe the result, looping until it has enough information (the classic **ReAct** pattern: Reason → Act → Observe). The modern, recommended way to build agents is with **LangGraph's** prebuilt agent constructor rather than the older `AgentExecutor` class.

```
from langgraph.prebuilt import create_react_agent

agent = create_react_agent(llm, tools=[get_weather])

result = agent.invoke({
    "messages": [("human", "Should I carry an umbrella in Hyderabad today?")]
})
print(result["messages"][-1].content)
```

6.3 LangGraph — stateful, multi-step orchestration

LCEL chains are great for linear pipelines, but many real applications need **branches, loops, and shared state** — e.g. an agent that retries, a multi-agent system, or a workflow with human-in-the-loop approval. **LangGraph** models your application as a graph of nodes (steps) and edges (control flow) operating over a shared state object.

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class State(TypedDict):
    question: str
    answer: str

def generate(state: State) -> State:
    resp = llm.invoke(state["question"])
    return {"answer": resp.content}

graph = StateGraph(State)
graph.add_node("generate", generate)
graph.add_edge(START, "generate")
graph.add_edge("generate", END)

app = graph.compile()
print(app.invoke({"question": "What is LCEL?"}))
```

TIP

Think of it this way: **LCEL** = compose a straight-line pipe. **LangGraph** = compose a flowchart with state, branches, and loops. Agents, multi-agent systems, and anything with retries or human approval steps belong in LangGraph.

6.4 Streaming & Async

Every Runnable supports streaming tokens as they're generated (better perceived latency in a UI) and async execution (better throughput for many concurrent requests).

```
for chunk in rag_chain.stream("Summarize chapter 2"):
    print(chunk, end="", flush=True)

# Async version for web servers (FastAPI, etc.)
result = await rag_chain.ainvoke("Summarize chapter 2")
```

6.5 LangSmith — observability, tracing & evaluation

Once a chain has multiple steps, retries, and tool calls, debugging blind is painful. **LangSmith** traces every step of every run (inputs, outputs, latency, token usage), and lets you build evaluation datasets to catch regressions before they reach production.

```
# .env
LANGSMITH_TRACING=true
LANGSMITH_API_KEY=ls___...
LANGSMITH_PROJECT=my-rag-app

# That's it — every invoke()/stream() call is now traced automatically
# and visible in the LangSmith dashboard.
```

6.6 Deployment

- **LangServe** — wraps a Runnable as a FastAPI REST endpoint with a couple of lines of code; good for quickly exposing a single chain as an API.

- **LangGraph Platform** — managed hosting/deployment for LangGraph applications with built-in persistence, streaming, and human-in-the-loop support.
- **Roll your own** — since every chain/agent is just a Python object with `invoke/ainvoke`, it drops cleanly into any FastAPI, Flask, or serverless function.

7. Putting It Together: A Minimal RAG Chatbot

This single, self-contained example threads together everything from Sections 4–6: loading a document, chunking it, embedding it, retrieving relevant chunks, and answering with memory of the conversation.

```

from dotenv import load_dotenv
load_dotenv()

from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_chroma import Chroma
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough
from langchain_core.chat_history import InMemoryChatMessageHistory
from langchain_core.runnables.history import RunnableWithMessageHistory

# 1. Load & split
docs = PyPDFLoader("company_handbook.pdf").load()
chunks = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=150).split_documents(docs)

# 2. Embed & index
vectorstore = Chroma.from_documents(chunks, embedding=OpenAIEmbeddings())
retriever = vectorstore.as_retriever(search_kwargs={"k": 4})

# 3. Build the RAG chain
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
prompt = ChatPromptTemplate.from_messages([
    ("system", "Answer using only the provided context. If unsure, say you don't know.\n\nContext: {context}"),
    ("placeholder", "{history}"),
    ("human", "{question}"),
])

def format_docs(docs):
    return "\n\n".join(d.page_content for d in docs)

rag_chain = (
    RunnablePassthrough.assign(context=lambda x: format_docs(retriever.invoke(x["question"])))
    | prompt
    | llm
    | StrOutputParser()
)

# 4. Add conversational memory
store = {}
def get_history(session_id):
    return store.setdefault(session_id, InMemoryChatMessageHistory())

chatbot = RunnableWithMessageHistory(
    rag_chain, get_history,
    input_messages_key="question", history_messages_key="history",
)

# 5. Chat
answer = chatbot.invoke(
    {"question": "How many vacation days do new hires get?"},
    config={"configurable": {"session_id": "demo-user"}},
)
print(answer)

```

8. Best Practices & Common Pitfalls

8.1 Do

- Pin your dependency versions (`langchain==x.y.z`) — the ecosystem moves fast.
- Prefer official partner packages (`langchain-openai`, etc.) over `langchain-community` equivalents when both exist — they're better maintained.
- Turn on LangSmith tracing as soon as a chain has more than 2 steps.
- Set explicit `chunk_size/chunk_overlap` and actually inspect a few retrieved chunks — bad retrieval is the #1 cause of bad RAG answers.
- Use Pydantic output schemas for anything downstream code will parse — don't regex an LLM's text.
- Set `temperature=0` for tasks with a single correct answer (extraction, classification, RAG QA).

8.2 Avoid

- Mixing very old (pre-0.1) LangChain tutorials with current code — the API changed substantially with the introduction of LCEL; old-style `LLMChain/ConversationChain` classes are legacy.
- Building custom agent loops by hand when LangGraph's prebuilt agent already covers the pattern.
- Sending your entire document corpus in the prompt instead of retrieving only relevant chunks.
- Ignoring token costs — log token usage during development, not just after the bill arrives.
- Hard-coding API keys in source files.

9. Learning Roadmap

Week 1 — Foundations

- Install LangChain; make a single ChatModel call.
- Learn PromptTemplate, StrOutputParser, and the LCEL pipe operator.
- Build a 3-step chain (prompt → model → parser).

Week 2 — Working with data

- Load a PDF/website with a document loader; split it into chunks.
- Generate embeddings and store them in a local vector store (Chroma/FAISS).
- Build your first RAG pipeline and evaluate answer quality by hand.

Week 3 — Statefulness

- Add conversational memory with RunnableWithMessageHistory.
- Add structured output with Pydantic schemas.
- Turn on LangSmith tracing and read a few traces end-to-end.

Week 4 — Agents & beyond

- Write a custom tool; bind it to a model; inspect tool_calls.
- Build a ReAct agent with LangGraph's create_react_agent.
- Model a small multi-step workflow as a LangGraph StateGraph.
- Deploy a chain with LangServe or a plain FastAPI endpoint.

10. Glossary

Runnable

The common interface (invoke/batch/stream/ainvoke) implemented by almost every LangChain component — prompts, models, retrievers, parsers, and chains.

LCEL

LangChain Expression Language — the ``|`` pipe syntax for composing Runnables into chains.

Chain

Any sequence of steps that transforms an input into an output; in modern LangChain, usually an LCEL pipeline rather than a ``Chain`` subclass.

Embedding

A numeric vector representation of text that captures semantic meaning, used for similarity search.

Vector store

A database optimized for storing embeddings and finding the nearest ones to a query vector (e.g. Chroma, FAISS, Pinecone).

Retriever

A component that fetches relevant documents/chunks for a query, typically backed by a vector store.

RAG

Retrieval-Augmented Generation — answering questions by retrieving relevant context and feeding it to the model, instead of relying only on the model's trained knowledge.

Tool

A function the model can choose to invoke (search, calculator, API call, ...) via native tool/function calling.

Agent

A model-driven control loop that decides which tools to call and when, until it can produce a final answer.

LangGraph

A library for building stateful, graph-structured LLM applications with branches, loops, and shared state — the modern foundation for agents.

LangSmith

LangChain's observability and evaluation platform: tracing, debugging, and dataset-based testing.

LangServe

A helper for exposing a Runnable as a REST API via FastAPI.

Context window

The maximum number of tokens a model can process in a single call (prompt + history + retrieved context + response).

11. Further Resources

Once comfortable with this guide, deepen your knowledge with:

- Official LangChain documentation (python.langchain.com) — the authoritative, always-current reference.
- LangGraph documentation — for anything involving agents or multi-step stateful workflows.
- LangSmith documentation — for tracing, debugging, and evaluation once you move past prototypes.
- Your model provider's own docs (OpenAI/Anthropic/Google) — for provider-specific features like structured outputs, vision, or prompt caching that LangChain wraps but doesn't hide.
- Read source code of `langchain-core`'s`runnables`` module once — it demystifies how ``|`` composition actually works under the hood.

This guide reflects LangChain's modern (LCEL + LangGraph) patterns. The framework evolves quickly — always cross-check version-specific details against the official docs for the exact versions pinned in your project.